Réponse TP3

Question 1

- **Nombre de nœuds :** 3 (`cass1`, `cass2` et `cass3`).
- **État :** les 3 sont en `UN` (Up, Normal) : ils sont démarrés, joignables et font pleinement partie du cluster.
- **Datacenter :** tous dans `dc1`.
- **Racks :** un nœud par rack. cass1 est dans rack1, cass2 dans rack2 et cass3 dans rack3.

Question 2 - Architecture obtenue

Le cluster `tp2-cluster` compte 3 nœuds Cassandra 4.1 lancés avec Docker Compose, sur le réseau `cass-net`. Ils sont tous dans un seul datacenter, `dc1`, et chacun est dans son propre rack :

```
Cluster tp2-cluster
        |
       dc1
  ┌──────┼──────┐
rack1  rack2  rack3
  |      |      |
cass1  cass2  cass3
```

- Les nœuds sont égaux (architecture peer-to-peer), sans chef. Seul `cass1` expose le port 9042 vers ma machine, et il sert de point d'entrée.
- `cass1` est le *seed* : les autres nœuds le contactent pour rejoindre le cluster.
- Le `GossipingPropertyFileSnitch` permet à chaque nœud de déclarer son datacenter et son rack aux autres.
- Chaque nœud a 16 tokens (`vnodes`), et les données sont réparties entre eux.
- Le keyspace utilise `NetworkTopologyStrategy` avec `'dc1': 3` : chaque donnée est copiée sur les 3 nœuds.

Cluster, datacenter, rack, node

Terme	Définition	Dans ce TP
Cluster	L'ensemble des nœuds qui forment une même base de données, identifiés par un même nom. C'est le niveau le plus large.	`tp2-cluster`
Datacenter	Un groupe logique de nœuds, en général un site géographique ou une région cloud. Le facteur de réplication se règle par datacenter.	`dc1`
Rack	Un groupe de nœuds qui partagent un même risque de panne (baie, alimentation, zone de disponibilité). Cassandra répartit les copies sur des racks différents.	`rack1`, `rack2`, `rack3`
Node	Une instance Cassandra, c'est-à-dire une machine (ici un conteneur) qui stocke une partie des données et répond aux requêtes. C'est le niveau le plus fin.	`cass1`, `cass2`, `cass3`

Question 3 -

```
DESCRIBE KEYSPACES;
DESCRIBE KEYSPACE meteo;

USE meteo;
DESCRIBE TABLES;
DESCRIBE TABLE releves_temperature;
```

```
SELECT * FROM releves_temperature WHERE ville_id = 'paris';
SELECT COUNT(*) FROM releves_temperature WHERE ville_id = 'paris';
```

```
cqlsh:meteo> DESCRIBE TABLE releves_temperature;

CREATE TABLE meteo.releves_temperature (
    ville_id text,
    observed_at timestamp,
    temperature_2m double,
    PRIMARY KEY (ville_id, observed_at)
) WITH CLUSTERING ORDER BY (observed_at ASC)
```

```
cqlsh:meteo> SELECT * FROM releves_temperature WHERE ville_id = 'paris';

 ville_id | observed_at                      | temperature_2m
----------+----------------------------------+----------------
    paris | 2026-10-06 12:00:00.000000+0000 |           22.2
    paris | 2026-10-06 13:00:00.000000+0000 |           23.5
    paris | 2026-10-06 14:00:00.000000+0000 |           24.2
    paris | 2026-10-06 15:00:00.000000+0000 |           24.2
    paris | 2026-10-06 16:00:00.000000+0000 |           24.1
    paris | 2026-10-06 17:00:00.000000+0000 |           23.4
```

Keyspace : meteo (NetworkTopologyStrategy, 'dc1': 3).

- **Table :** releves_temperature.
- **Principales colonnes :**
  - ville_id (text) : identifiant de la ville ;
  - observed_at (timestamp) : heure du relevé ;
  - temperature_2m (double) : température en °C à 2 m.
- **Partition key :** ville_id. Tous les relevés d'une ville sont stockés ensemble.
- **Clustering key :** observed_at. Elle distingue les relevés d'une même ville et les trie par heure.

Question 4 :

Partitionnement : on découpe les données en morceaux (les partitions). Chez moi, toutes les lignes d'une même ville forment un morceau. Le partitionnement répond à « dans quel morceau va cette ligne ? ».

Réplication : on fait plusieurs copies de chaque morceau, sur des nœuds différents. Avec RF = 3, mon morceau « paris » existe en 3 exemplaires, un par nœud. La réplication répond à « combien de copies, et où ? ».

RF = 3 veut dire que chaque partition est copiée **3 fois**, une fois sur chaque nœud.

Chez moi, la partition « paris » existe sur cass1, cass2 et cass3. Si un nœud tombe, les deux autres ont encore toutes les données.

Question 5

1. **Partition key** : pour un relevé de releves_temperature, c'est ville_id, par exemple 'paris'. Tous les relevés de Paris ont la même partition key, donc ils sont traités ensemble.
2. **Hash** : Cassandra applique une fonction de hachage (Murmur3) à cette valeur, ce qui donne toujours le même résultat pour 'paris'.

3. **Token** : le résultat du hash est un grand nombre, le token (entre environ $-9 \cdot 10^{18}$ et $+9 \cdot 10^{18}$). Il désigne un point sur l'anneau de tokens.
4. **Nœud(s) responsable(s)** : chaque nœud possède des plages de tokens (16 chacun). Le token de 'paris' tombe dans la plage d'un nœud, qui devient le premier réplica.
5. **Réplicas** : avec RF = 3, Cassandra continue sur l'anneau pour choisir les copies suivantes, sur des nœuds d'autres racks. Ici, comme il y a 3 nœuds, les réplicas sont cass1, cass2 et cass3.

En résumé : 'paris' est transformée en token, le token désigne un nœud, et RF = 3 ajoute deux copies sur les autres nœuds.

```
PS C:\Users\Thoma\Documents\09_SupDeVinci\M2\Données Dist\T3> docker exec cass1 nodetool
getendpoints meteo releves_temperature paris
172.23.0.4
172.23.0.2
172.23.0.3
```

En faisant le getendpoints j’ai accès à l’ip des trois noeuds qui possèdent les données

Question 6

One - On attends seulement un réplica, donc même si deux noeuds sont en panne, ça tourne, au risque de redonner une donnée en retard

```
cqlsh:meteo> CONSISTENCY ONE;
Consistency level set to ONE.
cqlsh:meteo> SELECT * FROM releves_temperature
         ... WHERE ville_id = 'paris' AND observed_at = '2026-10-07 07:00:00.000000+0000';

 ville_id | observed_at                      | temperature_2m
----------+----------------------------------+----------------
    paris | 2026-10-07 07:00:00.000000+0000 |           14.5

(1 rows)
```

QUORUM - On attends RF/2 + 1 réplica pour être équilibré, dans mon cas de figure ça plante si deux noeuds sont en panne

```
cqlsh:meteo> CONSISTENCY QUORUM;
Consistency level set to QUORUM.
cqlsh:meteo> SELECT * FROM releves_temperature WHERE ville_id = 'paris' AND observed_at = '2026-10-07 07:
00:00.000000+0000';

 ville_id | observed_at                      | temperature_2m
----------+----------------------------------+----------------
    paris | 2026-10-07 07:00:00.000000+0000 |           14.5

(1 rows)
```

ALL - On a besoin de tous les noeuds, c'est forcément le plus fiable mais ça plante si j'ai un seul noeud en panne

```
cqlsh:meteo> CONSISTENCY ALL;
Consistency level set to ALL.
cqlsh:meteo> SELECT * FROM releves_temperature WHERE ville_id = 'paris' AND observed_at = '2026-10-07 07:
00:00.000000+0000';

 ville_id | observed_at                      | temperature_2m
----------+----------------------------------+----------------
    paris | 2026-10-07 07:00:00.000000+0000 |           14.5

(1 rows)
```

Forcément tout fonctionne car on a pas fait de simulation d’échecs

Question 7

```
PS C:\Users\Thoma\Documents\09_SupDeVinci\M2\Données Dist\T3> docker stop cass3
cass3
PS C:\Users\Thoma\Documents\09_SupDeVinci\M2\Données Dist\T3> docker exec cass1 nodetool status
Datacenter: dc1
===============
Status=Up/Down
|/ State=Normal/Leaving/Joining/Moving
--  Address     Load        Tokens  Owns (effective)  Host ID                               Rack
UN  172.23.0.3  238.09 KiB  16      100.0%            ed2ba4ec-708a-42af-82f1-1a2b191f3ccb  rack2
DN  172.23.0.4  191.25 KiB  16      100.0%            43860a9e-e3dc-4739-90cd-fd7c1bc39096  rack3
UN  172.23.0.2  177.56 KiB  16      100.0%            085b0122-8038-4764-b790-8718a95764c3  rack1
```

Le cluster cass3 a été stoppé, on le voit à son état DN, les deux autres tournent toujours.

Question 8

```
cqlsh:meteo> CONSISTENCY ONE;
Consistency level set to ONE.
cqlsh:meteo> SELECT * FROM releves_temperature WHERE ville_id = 'paris' AND observed_at = '2026-10-07 07:00:00.000000+0000';

 ville_id | observed_at                     | temperature_2m
----------+---------------------------------+----------------
    paris | 2026-10-07 07:00:00.000000+0000 |           14.5

(1 rows)
```

En consistency one, la requête fonctionne sans problèmes, même si le noeud cass3 est en panne

```
cqlsh:meteo> CONSISTENCY QUORUM;
Consistency level set to QUORUM.
cqlsh:meteo> SELECT * FROM releves_temperature WHERE ville_id = 'paris' AND observed_at = '2026-10-07 07:00:00.000000+0000';

 ville_id | observed_at                     | temperature_2m
----------+---------------------------------+----------------
    paris | 2026-10-07 07:00:00.000000+0000 |           14.5

(1 rows)
```

En consistency quorum, la requête tourne aussi, RF/2 + 1 = 3/2+1 = 2.5 = 2, il y a 2 noeuds actifs donc ça fonctionne

```
cqlsh:meteo> CONSISTENCY ALL;
Consistency level set to ALL.
cqlsh:meteo> SELECT * FROM releves_temperature WHERE ville_id = 'paris' AND observed_at = '2026-10-07 07:00:00.000000+0000';
NoHostAvailable: ('Unable to complete the operation against any hosts', {<Host: 127.0.0.1:9042 dc1>: Unavailable('Error from serv
er: code=1000 [Unavailable exception] message="Cannot achieve consistency level ALL" info={\'consistency\': \'ALL\', \'required_r
eplicas\': 3, \'alive_replicas\': 2}')})
```

En consistency All, vu qu'un noeud est en panne, nous avons une erreur et la requête ne s'exécute pas.

En résumé, le RF fixe le nombre total de copies. Comme il y a 3 nœuds, chacun possède une copie de chaque partition. Quand un nœud tombe, il en reste donc 2 : c'est ce qui permet à ONE et QUORUM de répondre quand même. Avec RF = 1, la panne du nœud qui porte la donnée la rendrait indisponible, quel que soit le niveau.

Question 9

Le retour de cass3 ne pose aucun problème, les noeuds tournent tous et un test en consistency ALL le prouve, tout est OK.

Pour un test, pendant que cass3 était coupé j'ai ajouté une ligne via INSERT INTO, puis après avoir rouvert cass3, je me connecte dessus

```
PS C:\Users\Thoma\Documents\09_SupDeVinci\M2\Données Dist\T3> docker exec -it cass3 cqlsh
```

Puis je fais mon SELECT *

```
paris | 2026-10-07 07:00:00.000000+0000 |           14.5
paris | 2026-10-20 12:00:00.000000+0000 |           99.9
```

On retrouve bien ma ligne, la réplication fonctionne

Question 10

Donnée : j'ai inséré un relevé (ville_id = 'paris', observed_at = '2026-10-20 12:00:00', temperature_2m = 99.9) dans la table releves_temperature, pendant que cass3 était arrêté.

Partition : ce relevé appartient à la partition paris, car sa partition key ville_id vaut 'paris'. Cassandra en déduit un token, qui désigne les nœuds responsables.

Réplication : avec RF = 3, la partition paris est copiée sur cass1, cass2 et cass3.

Panne d'un nœud : cass3 est arrêté, il ne reste que 2 réplicas sur 3. nodetool status affiche cass3 en DN.

Données toujours accessibles : les lectures en ONE et QUORUM fonctionnent, et l'écriture en QUORUM réussit, car 2 réplicas suffisent. Seul ALL échoue, puisqu'il exige les 3.

Retour du nœud : après docker start cass3, il repasse en UN. J'observe que le relevé inséré pendant la panne est présent sur cass3, alors qu'il était éteint au moment de l'écriture. Les autres nœuds ont gardé l'écriture de côté (*hinted handoff*) et l'ont transmise à cass3 à son retour, ce qui remet les 3 copies d'accord.

Conclusion : grâce à la réplication, la panne d'un nœud n'a pas interrompu le service ni fait perdre de données, et le nœud a rattrapé son retard tout seul en revenant.

Question 11 - Synthèse

Mon cluster tp2-cluster compte 3 nœuds (cass1, cass2, cass3) dans dc1, un par rack. Le keyspace meteo utilise NetworkTopologyStrategy avec RF = 3 : chaque partition de releves_temperature (par exemple paris) est donc copiée sur les 3 nœuds.

J'ai arrêté cass3, et nodetool status l'a affiché en DN. Il restait 2 nœuds disponibles, donc 2 réplicas sur 3. J'ai alors testé les trois niveaux de cohérence :

- ONE (1 réplica exigé) : fonctionne.
- QUORUM (2 exigés) : fonctionne.
- ALL (3 exigés) : échoue, car il n'y a plus que 2 réplicas.

J'ai aussi inséré un relevé de test pendant la panne, en QUORUM, et l'écriture a réussi.

Après docker start cass3, le nœud est repassé en UN. J'ai vérifié que le relevé inséré pendant la panne était bien présent chez lui : les autres nœuds l'ont retransmis à son retour, et les 3 copies sont de nouveau identiques.

Grâce à la réplication, chaque donnée existe en plusieurs exemplaires sur des nœuds différents. Quand cass3 tombe, les deux autres nœuds ont encore une copie complète de toutes les partitions : aucune donnée n'est perdue ni inaccessible.

Il n'y a pas non plus de nœud principal à remplacer, car tous les nœuds sont égaux et n'importe lequel peut répondre. Tant qu'il reste assez de réplicas disponibles pour le niveau de cohérence demandé (1 pour ONE, 2 pour QUORUM), les lectures et les écritures continuent, et le nœud en panne rattrape son retard à son retour.